

Tech Stack Writeup for CogniFlow

Crew: Animatrix

Stage: Week 1 Submission

To ensure CogniFlow is fast, highly responsive, and completely sustainable, our crew has designed a decentralized system architecture. Every tool selected satisfies two strict principles: it must provide a reliable foundation for complex visual processing, and it must operate within a permanent, zero-cost developmental free tier. Below is the breakdown of the technologies proposed to power the platform.

1. The User Interface & Animation Layer

This layer handles everything the student interacts with on their screen, ensuring transitions are smooth and text-to-visual workflows load instantaneously on both desktops and mobile browsers.

Next.js (React) & TypeScript

Next.js would serve as the core framework for our user interface. It ensures pages load quickly by optimizing how code is shipped to the browser.

We're using TypeScript as a **structural guardrail** for CogniFlow. Because our app translates raw textbook text into dynamic code animations, the data flowing between our AI and our frontend has to be flawless. TypeScript acts like an automated referee—it catches typos, broken variables, and misaligned data structures while we are coding, long before it ever reaches a student's screen.

These would be hosted on Vercel (Free Tier).

Framer Motion

Instead of relying on slow cloud video-generation tools, CogniFlow would draw animations directly inside the student's browser using pure code. Framer Motion is an open-source library that reads structured text roadmaps generated by our AI and translates them into smooth, hardware-accelerated shapes, lines, and movements in real-time. It is under the MIT License and completely free.

2. The Backend & Processing Layer

This would be the brain of the platform, interpreting uploaded classroom materials and securely running the artificial intelligence pipelines we would work with.

FastAPI (Python)

FastAPI would serve as our secure traffic controller. Since it is built to handle multiple connection streams simultaneously without lagging, it will act as a high-speed middleman. It would securely relay instructions between the user's browser, our file database, and our heavy AI models while keeping private credentials hidden safely behind a digital firewall. Using Python file-parsing tools, it will extract the text layer from the files uploaded to the cloud storage.

It would be deployed on Render (Free Tier).

Google Gemini API (via Google AI Studio)

Gemini forms the intellectual engine of CogniFlow. It processes large chunks of complex textual knowledge (like a multi-page high school biology chapter or an advanced engineering note) and filters the data to output an organized, machine-readable animation timeline. Additionally, it would handle natural language speech synthesis to run our interactive, conversational audio tutoring loops.

The Google Gemini API is our choice since it was built as a multimodal model to handles diverse tasks. It will function as the reasoning engine which handles the breakdown of the uploaded lesson into structured explanation, as well as analyze the uploaded note. It would also operate on a JSON mode to speak with the Framer Motion tool, acting as the visualization explanation agent.

Moreover, it would handle the Text-to-Speech and Speech-to-Text, using the Audio Live API feature, so we would not need multiple APIs for these tasks.

3. The Storage, Security, & Account Layer

This layer acts as our safe repository, ensuring student profiles are secured, test logs are maintained, and heavy files are isolated from the main server application.

Google Firebase

To keep our system lightweight and unified, we are deploying Firebase as an all-in-one background suite. Cloud Firestore would act as our text database, storing lesson blueprints as flexible documents that mirror our AI's native output. Firebase Authentication manages secure email and Google student sign-ins. Cloud Storage isolates heavy physical items like student textbook PDFs and image uploads so they never clog up our processing tracks.

We would use the Firebase Spark Plan, which will cost us nothing.